ناطق · NAATIQ — DOCUMENTATION

Setup and Runbook

How to deploy Naatiq from scratch, and how to diagnose it when something breaks

Part One — Setup

1. Supabase

Create a project, then apply the four migrations in order from `supabase/migrations/`:

Migration	Creates
`0001_initial_schema.sql`	`users`, `profiles`, `conversations`, `matches`, indexes, RLS enabled
`0002_dashboard_views.sql`	`first_name` generated column, column-level grants, the three `security_invoker` views
`0003_public_cvs_and_cv_links.sql`	Public `cvs` bucket, `cv_pdf_path` on `dashboard_profiles`
`0004_placements_and_public_stats.sql`	`placements` table, count-only anon exposure, updated `dashboard_stats`

Then create a Storage bucket named **`cvs`**.

2. Gotenberg

Deploy `infra/gotenberg/` to Railway from the Dockerfile — it binds Gotenberg to Railway's injected `$PORT`. Copy the resulting public URL; it becomes `GOTENBERG_URL`.

Verify it is alive:

```
curl -I https://<your-gotenberg-host>/health
```

3. Twilio WhatsApp Sandbox

1. In the Twilio console, open **Messaging → Try it out → Send a WhatsApp message**.
2. Note the sandbox number (typically `+1 415 523 8886`) and the join code.

3. Set the **"When a message comes in"** webhook to the deployed webhook URL, method **POST**: `https://<project-ref>.functions.supabase.co/whatsapp-webhook`
4. From any phone, WhatsApp the sandbox number with `join <your-code>` to opt in. For this deployment the code is **`join reach-sheep`**.

The join code also belongs in the landing page CONFIG block in `web/index.html`, where it pre-fills the "Start on WhatsApp" button so a first-time visitor opts in without having to be told the code. Sandbox codes are rotated by Twilio — if the CTA stops working, this is the first thing to re-check.

4. API keys

Obtain an Anthropic API key, a Groq API key, and — only if voice replies are wanted on a paid plan — an ElevenLabs key and voice ID.

5. Function secrets

Set all of these in Supabase → Edge Functions → Secrets:

```
SUPABASE_URL=https://<project-ref>.supabase.co
SUPABASE_SERVICE_ROLE_KEY=<service role key>

TWILIO_ACCOUNT_SID=<sid>
TWILIO_AUTH_TOKEN=<token>
TWILIO_WHATSAPP_FROM=whatsapp:+14155238886      # the whatsapp: prefix is REQUIRED

GROQ_API_KEY=<key>
ANTHROPIC_API_KEY=<key>
GOTENBERG_URL=https://<gotenberg-host>          # include the scheme

ELEVENLABS_API_KEY=<key>
ELEVENLABS_VOICE_ID=<voice id>
WATHEFTI_VOICE_REPLIES=false                    # true only on a paid ElevenLabs plan

WATHEFTI_SELFTEST_SECRET=<a long random string>
```

Two settings cause most first-deploy failures: a `TWILIO_WHATSAPP_FROM` without the `whatsapp:` prefix (Twilio error 63007), and a `GOTENBERG_URL` without `https://`.

6. Deploy the functions

Function	`verify_jwt`	Why
`whatsapp-webhook`	`false`	Twilio cannot send a Supabase JWT
`generate-cv`	`true`	Invoked only server-to-server

When **updating** an existing function, pass `import_map_path: "deno.json"` explicitly or the deploy fails on a missing import map.

7. Front end

`dashboard/index.html`, `web/index.html` and `web/employers.html` are self-contained. Open them directly or serve them from any static host. Each needs the project URL and the **publishable** key — never the service role key. Set `WHATSAPP_NUMBER` and `JOIN_CODE` in the landing page CONFIG block.

8. Verify the installation

```
curl "https://<project-ref>.functions.supabase.co/whatsapp-webhook?selftest=<secret>"
```

Every secret should report as present, and `twilio_from`, `gotenberg_url` and `elevenlabs_voice_id` should show the expected values. Then send a WhatsApp voice note and expect an Arabic reply within a few seconds.

Part Two — Runbook

The self-test endpoint

Supabase's log API shows request lines but not `console.error` output, so this endpoint is the primary diagnostic tool. **Start here for any failure.**

Query	Tests
`?selftest=<secret>`	Every env var's presence and the resolved config
`?selftest=<secret>&claude=1`	A live round trip through Claude
`?selftest=<secret>&sendto=<number>`	A live outbound WhatsApp message
`?selftest=<secret>&ttstest=1`	The ElevenLabs path
`?selftest=<secret>&cvtest=<user_id>`	CV generation for an existing user

For the CV pipeline specifically, invoke `generate-cv` with `{"user_id": "...", "debug": true}` to run it synchronously and receive a `steps[]` array showing exactly where it succeeded or failed.

Symptom → cause

Nothing happens when I send a WhatsApp message. Check the Twilio console for webhook delivery errors first — if Twilio never reached the function, the problem is the webhook URL or the HTTP method. Then confirm the phone has joined the sandbox (the join code expires) and that `verify_jwt` is `false` on the webhook.

I get replies, but no CV arrives. Run `&cvtest=<user_id>`, or invoke `generate-cv` in debug mode and read `steps[]`. The usual causes are a Gotenberg URL missing its scheme, a Gotenberg instance that has gone to sleep, or a missing `cvs` bucket.

Replies are generated but never arrive — Twilio error 63038. `Account ... exceeded the 50 daily messages limit`. Twilio **trial** accounts cap outbound at 50 messages per day, and every media message (each delivered CV) counts. The agent will look completely healthy — transcription, interview and reply all succeed — while nothing reaches the user. The cap resets on Twilio's daily cycle; upgrading the account out of trial removes it. **This is a demo-day risk: a heavy morning of testing can silently consume the day's quota.**

To see delivery failures directly:

```
select * from delivery_failures;    -- last 24h, newest first
```

or hit `?selftest=<secret>`, which reports `recent_delivery_failures` inline.

Twilio error 63007. `TWILIO_WHATSAPP_FROM` is not a WhatsApp-enabled address. It must be exactly `whatsapp:+14155238886`.

Anthropic returns 400 "temperature is deprecated for this model". The `temperature` parameter reached the API. Claude Sonnet rejects it; remove it from the request body.

ElevenLabs returns 402. The free plan does not allow API access to the voice library. No voice ID fixes this. Keep `WATHEFTI_VOICE_REPLIES=false`, or upgrade the plan and set it to `true`.

The dashboard shows demo data or "failed to fetch". The Supabase URL or publishable key is wrong, or the views are missing. The dashboard is designed to fall back to embedded snapshot data rather than show an error, so stale numbers are the expected failure mode. Confirm by querying a view directly with the publishable key.

Supabase advisor flags `security_definer_view`. A view was created without `security_invoker = true`. Recreate it with that option. Note that `create or replace view` cannot reorder columns — drop and recreate.

A function deploy is rejected. Check for stray characters at the end of a file, an oversized or truncated payload, and a missing `import_map_path: "deno.json"`.

Resetting for a clean demo

Delete rows in dependency order — `conversations`, `matches`, `placements`, `profiles`, then `users` — and clear the `cvs` bucket. Re-run one full conversation from a real phone afterwards to confirm the pipeline still completes end to end.

Regenerating the documentation

The Markdown in `docs/src/` is the source of truth; PDFs are build artefacts and should never be edited by hand.

```
pip install markdown weasyprint pillow --break-system-packages
python3 docs/build_docs.py
```